

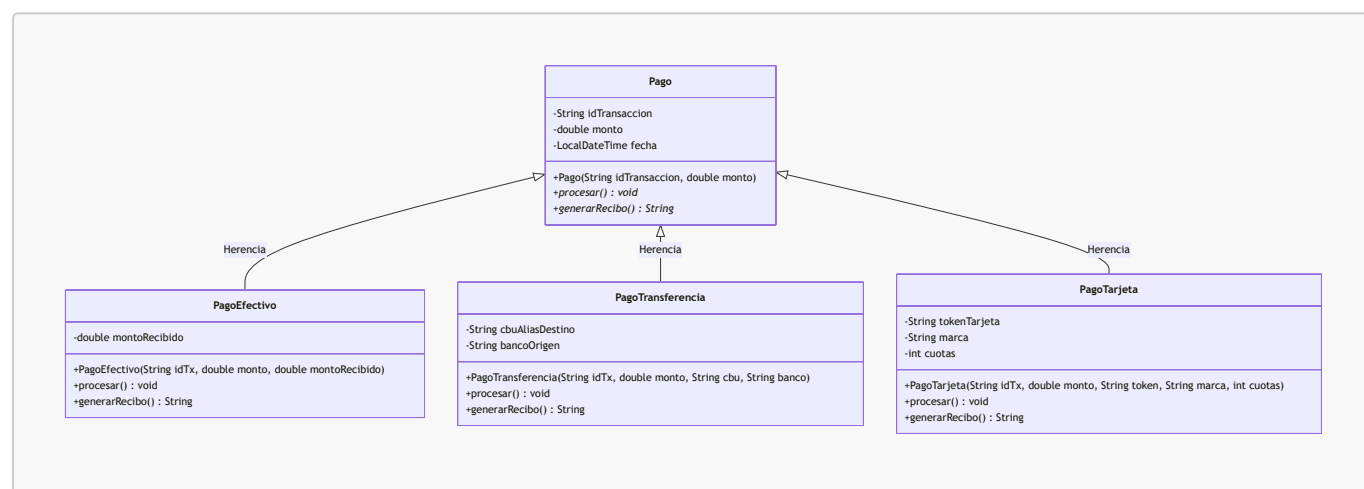
Laboratorio Práctico sobre Colecciones, procesamiento CSV

El presente laboratorio es para realizar la práctica correspondiente, encarando los temas que van a ser parte del parcial.

El objetivo final es completar un ejercicio de punta a punta, definiendo jerarquias de herencia para luego usar esa jerarquia para importar un archivo CSV en memoria y realizar algunos puntos que permitan procesamiento de esos datos colocados en una coleccion. A teclear....

Enunciado

Una empresa que comercializa bienes y servicio pidio un modulo que procese los pagos a los cuales le ha delegado a un entidad de cobros rapidos. Para ello se hizo un análisis y se definio la siguiente estructura de objetos.



Pasos a realizar de configuracion inicial

1. Crear el proyecto de JAVA usando el siguiente comando de maven

```
mvn archetype:generate "-DgroupId=com.frc.utn.isi.backend.pagos" -
DartifactId="pagos-app" -DarchetypeGroupId="org.apache.maven.archetypes" -
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

2. Implementar la jerarquía propuesta, que va a soportar todo el manejo de los pagos en memoria para obtener una serie de reportes sobre los datos que luego se usaran.
3. Parte del proyecto requiere crear una clase adicional que aplique un estilo de patron Factory que permita crear los objetos correspondientes:

```
switch (tipoPago.toUpperCase()) {
    case "EFECTIVO":
        double recibido = (double) metadata.getOrDefault("montoRecibido", monto);
```

```

        return new PagoEfectivo(idTx, monto, recibido);

    case "TRANSFERENCIA":
        String cbu = (String) metadata.get("cbuAliasDestino");
        String banco = (String) metadata.get("bancoOrigen");
        return new PagoTransferencia(idTx, monto, cbu, banco);

    case "TARJETA":
        String token = (String) metadata.get("tokenTarjeta");
        String marca = (String) metadata.get("marca");
        int cuotas = (int) metadata.getOrDefault("cuotas", 1);
        return new PagoTarjeta(idTx, monto, token, marca, cuotas);

    default:
        throw new IllegalArgumentException("Medio de pago no soportado: " +
            tipoPago);
    }
}

```

4. Crear una nueva carpeta en el proyecto llamador resources, dentro de la misma crear una carpeta llamada data, como muestra la imagen.



5. Dentro de la carpeta, colocar el archivo **pago_importar.md** dentro de la carpeta data.
6. Los métodos polimórficos para cada uno, debería aplicar dicha lógica:
 1. Procesar
 1. Efectivo - se calcula el vuelto en caso de que el monto recibido sea mayor al monto del pago, si es inferior debe lanzar una excepción
 2. Transferencia - al monto del pago se le agrega un 0.035% sobre ese monto en concepto de costos bancarios
 3. Tarjeta - si el pago se hizo en mas de 3 cuotas se le agrega al monto un interes del 3% solo si las cuotas es mayor a 3
 2. generarRecibo
 1. Efecivo - Retorna todos los datos, incluyendo el cambio que se da y el valor neto que se cobro
 2. Transferencia - Retorna los datos mas el monto que se le agrega por costos bancarios
 3. Tarjeta - Retorna todos los datos mas el interes que se cobro si el mismo el mayor a cero solamente

Pasos a importar el CSV

Hasta ahora fueron cosas muy sencillas, armar un par de clases y en esta etapa vamos a implementar una clase que permita importar el archivo **pagos_importar.csv**, tenga en cuenta que la primera linea de ese archivo es de cabecera y no debería procesarse, de la siguiente manera:

```
public class PagosMapper {  
    private URL folderPath; // la carpeta donde se encuentra el archivo csv  
  
    public PagosMapper (URL folderPath) {  
        // Su codigo va aqui  
    }  
}
```

Dentro de esa clase se debera:

1. Agregar un metodo que a partir de **folderPath** lea el archivo csv y que lea cada linea para convertirla en un **Pago**, para leerlos usaremos un bufferedReader como se ejemplifica a continuacion.

```
Path archivo = Path.of(folderPath.URI);  
try (BufferedReader br = Files.newBufferedReader(archivo, StandardCharsets.UTF_8))  
{  
    // Su codigo va aqui  
}
```

Tenga en cuenta que para usar **Path.of()** en conjunto con **URL**, debes tener cuidado ya que Path.of() no acepta una URL directamente, sino un objeto URI. Además, la resolución de rutas relativas con ../ puede fallar. Por eso ponemos esa linea.

El atributo **folderPath** que se debe asignar cuando se cree el mapper debe ejecutar esta linea para que en definitiva se pueda realizar la importación

```
URL resourceUrl = App.class.getResource("../resources/data/pagos_importar.csv");
```

Esta linea carga en la variable **resourceUrl** un objeto que indica de donde se va a obtener el archivo, pero de ninguna manera es el archivo en si, para obtener y abrir el archivo para procesarlo se debe usar **Path.of**. Usted sabra donde colocar la linea que cargue memoria la ubicacion del recurso...

Por último debemos acotar que dicho metodo va a retornar una colección lineal, ya sea ArrayList o LinkedList, a preferencia de ustedes. Lo que yo podria proponer como retorno.

```
List<Pago>
```

2. Una vez hecho la función que lee todas las línea, no implica que las mismas se conviertan a un **Pago**, para eso usted debe crear un segundo método que a partir de una linea del archivo, usando la clase donde implemento el patrón Factory en la sección anterior, retorne un Pago. **Analice el archivo CSV** para determinar como es que se debe invocar, que campo es relevante para Factorizar un **Pago** de un determinado tipo concreto y retornarlo.

Ahora la cuestion es donde usariamos esta clase **PagosMapper**, pues veamos esto en la siguiente sección...

Los métodos que aplicaran reglas de negocio...

En esta seccion indicaremos como crear una clase que permita cargar, procesar y aplicar diferentes búsquedas y reportes de los datos cargados en memoria. Para ello se creara un clase que tenga la siguiente estructura:

```
public class Pagos {  
    private List<Pago> items;  
}
```

Esta clase va a contener su constructor o no, dependerá de como usted lo implemente. Pero si tendra los siguientes métodos:

1. Cargar todos los pagos, el mismo debe usar la clase **PagosMapper**, la cabecera de dicho método se podria definir de la siguiente manera:

```
public void importarPagos(URL folderPath) {  
    // Su codigo va aquí  
}
```

2. Determinar cual es el monto neto total que la empresa tendrá como ingreso de las ventas de sus bienes y servicios
3. Mostrar un listado con toda la informacion de los recibos generados, para aquellas transacciones cuyo monto original sea mayor al valor promedio de los pagos realizados.
4. Determinar la cantidad de pagos se hicieron en cada fecha, la firma del método seria:

```
public Map<LocalDateTime, Integer> cantidadPagosPorFecha() {  
    // Su codigo va aqui  
}
```

5. Determinar el monto total por cada tipo de pago que tiene cargados, la firma del metodo seria:

```
public Map<String, Double> totalPorTipoDePago {  
    // Su codigo va aqui  
}
```

Ayudita **objeto.getClass().getName()**

Cerrando el Laboratorio

Simple cree un main que llame y haga uso de todos los metodos de la seccion anterior.

A divertirse!!!!

